

Resource-Efficient TinyML Pipeline for Embedded Deployment on ESP32 Microcontroller

1 Introduction

The deployment of deep learning models onto resource-constrained embedded devices has garnered significant interest due to the proliferation of Internet of Things (IoT) systems and the demand for on-device intelligence. Conventional deep learning architectures typically require extensive computational resources, memory, and power, which are incompatible with the limitations of microcontroller units (MCUs). In contrast, Tiny Machine Learning (TinyML) seeks to enable machine learning inference directly on ultra-low-power devices through model optimization and efficient deployment techniques. TinyML has emerged as an essential paradigm for achieving real-time decision making, reduced latency, enhanced privacy, and energy efficiency in embedded AI systems.

However, converting high-capacity neural networks into formats suitable for deployment on microcontrollers involves non-trivial optimization processes, including quantization and architectural adaptation. This report investigates the end-to-end pipeline for transforming a high-level deep learning model into an optimized, quantized inference engine compatible with the ESP32 microcontroller, highlighting both practical methods and performance outcomes.

2 Methodology

2.1 Model Architecture and Training

A lightweight convolutional neural network based on the MobileNetV2 architecture was selected due to its balance between model expressiveness and computational efficiency. MobileNetV2 employs depthwise separable convolutions and inverted residual modules to reduce parameter count and computational complexity, which is well-suited for constrained environments such as MCUs. The model was trained using the PyTorch framework on an image classification dataset, with input images resized to 100×100 pixels and normalized using standard ImageNet statistics. A width multiplier of 0.35 was applied to further reduce the number of trainable parameters and overall computational load.

Training was conducted using stochastic gradient descent with an adaptive learning rate strategy, optimizing cross-entropy loss over multiple epochs. The dataset was partitioned into training, validation, and test subsets, and model performance was evaluated iteratively to ensure convergence and generalization.

2.2 Conversion Workflow

Following training, the model conversion pipeline involved multiple stages to generate a format suitable for embedded inference. First, the trained PyTorch model was exported to the ONNX (Open Neural Network Exchange) format, enabling interoperability between deep learning frameworks. Subsequently, the ONNX model was converted to a TensorFlow SavedModel using existing conversion tools, which supported further transformation into TensorFlow Lite (TFLite) format. This stage prepares the model for execution within the TensorFlow Lite Micro framework, which is specifically designed for resource-limited hardware.

2.3 Quantization Techniques

Quantization plays a pivotal role in reducing model size and computational demand by representing weights and activations using reduced precision formats, such as 8-bit integers (INT8), while maintaining acceptable accuracy levels. The conversion process was configured to produce three model variants: full precision (FP32), half precision (FP16), and fully quantized INT8 models. Representative samples from the training set were used to calibrate the quantization process, ensuring that distributional properties of activations were preserved during conversion.

Quantization offers several benefits, including significant reductions in model footprint (often more than 4× compared to FP32) and improved inference efficiency on MCU platforms, enabling models to fit within limited memory and run within tight execution budgets.

2.4 Edge Deployment on ESP32

The optimized INT8 TFLite model was converted into a C header file containing a byte array representation of the network, enabling integration with microcontroller firmware. This representation aligns with the TensorFlow Lite Micro inference engine, which executes the model directly on embedded hardware with minimal overhead. The ESP32 microcontroller, equipped with a dual-core Xtensa processor and modest RAM, serves as the target platform for this deployment. The C array was integrated into a microcontroller project and deployed using standard flashing tools.

3 Results

The different model variants exhibited expected trade-offs between model size and performance. The FP32 baseline achieved the highest classification accuracy but had a prohibitive memory footprint for embedded use. The FP16 model reduced storage requirements substantially, with modest accuracy degradation, while the INT8 quantized model achieved the smallest footprint and enabled feasible deployment on the ESP32.

Quantized models demonstrated up to ~4× reduction in model size compared to full precision, facilitating real-time inference within the constraints of the microcontroller’s memory budget. Although slight reductions in classification accuracy were observed, the performance remained suitable for practical applications. Moreover, the INT8 variant efficiently executed on the embedded platform with lower inference latency and reduced energy consumption relative to higher-precision counterparts.

4 Discussion

The principal trade-off in this pipeline is between model precision and resource efficiency. Quantization leads to reduced numerical precision, which can occasionally degrade classification performance; however, such trade-offs are necessary to achieve viable on-device execution. Furthermore, generating a representative calibration dataset was critical to maintaining model robustness post-quantization.

While traditional cloud-based inference systems offer higher accuracy due to unrestricted compute, on-device inference provides benefits in terms of latency, privacy, and network independence, which are essential in many real-time IoT scenarios. This report’s methodology aligns with best practices in TinyML deployments, balancing computational limitations with performance requirements.

5 Conclusion

This study demonstrates a comprehensive and practical pipeline for converting deep learning models into a format suitable for deployment on resource-constrained microcontrollers. Through efficient network design, multi-stage conversion, and aggressive quantization, it is possible to achieve deployable machine learning inference on devices such as the ESP32. These techniques pave the way for applications requiring localized intelligence, low latency, and low power consumption.

The outcomes emphasize the importance of optimization techniques in enabling TinyML systems and provide a framework that future research can build upon, particularly in areas such as adaptive models, continual learning on the edge, and cybersecurity integration in edge AI systems.